Programming Assignment 2 - Into GPGPU-Sim

1. Introduction

This assignment is meant to familiarize you with the simulator and serve as a warm up for PA3 and the final project. You will be delving into the codebase and modifying GPGPU-Sim. Use the same Docker environment that you used in PA1.

2. Task Description

You have 3 subtasks that you need to complete. The tasks are independent of each other. Therefore, when you finish one task, comment out the code you modified before you move on to the next task. You can also use different git branches for each task.

2.1. Instruction Semantics (Functional Simulator)

This task requires you to modify the functional simulator. You need to change the semantics of float(32-bits) add instruction to power operation. Floating-point add instruction performs fd = fa + fb. The registers fa and fb store 32-bits floating-point values. For example, fa=2.0 and fb=3.0 will produce 5.0, and write it to fd.

The modified instruction should generate the value $8 = 2.0^{3.0}$ and store it to fd. Use the vectorAdd.cu file shared on Moodle to test your code. The original instruction semantic should generate the results as follows.

```
source vector A: 1.000 2.000 3.000 4.000 5.000 6.000 7.000 8.000 9.000 1.000
source vector B: 1.000 1.000 2.000 3.000 4.000 5.000 6.000 6.000 7.000 8.000
result vector C: 2.000 3.000 5.000 7.000 9.000 11.000 13.000 14.000 16.000 9.000
```

The new instruction semantic should generate the following results,

```
source vector A: 1.000 2.000 3.000 4.000 5.000 6.000 7.000 8.000 9.000 1.000
source vector B: 1.000 1.000 2.000 3.000 4.000 5.000 6.000 6.000 7.000 8.000
result vector C: 1.000 2.000 9.000 64.000 625.000 7776.000 117649.000 262144.000 4782969.000 1.000
```

For this task, you will need to read and understand the code in instructions.cc file under cuda-sim directory.

2.2. Benchmark Performance Study (Timing Simulator)

In this task, you need to run the 5 benchmarks from ISPASS-2009 in GPGPU-Sim with SM6_TITANX and SM7_TITANV configurations, and plot IPCs of the benchmarks. This particular study is meant to acquaint you with the ISPASS benchmarks, as you will be using them in your final project. To limit the simulation time, the maximum number of instructions you should simulate is 1,000,000,00 for this task.

Download the compiled benchmarks from Moodle. Follow the instructions below to run the benchmarks,

- Create a ”test” directory in the container (mkdir test).
- Copy the benchmark folders to the ”test” directory.
- Copy the gpgpusim.config and *islip.icnt files to each benchmark directory.
- Run the benchmarks using the commands in “commands_for_running_ispass.txt”. Pay attention to the benchmark inputs. The input data files are provided with the benchmarks. Do not delete these files.

Plot IPCs for each configuration and benchmark. X-axis should be the benchmarks, Y-axis should be the IPCs. You should have two lines, one for SM6_TITANX and another for SM7_TITANV. Add a short discussion about your plots to your report.

2.3. Branch Instructions and Divergence (Timing Simulator)

In this last task, you need to modify the performance/timing simulator code by adding counters to report the following statistics. To limit the simulation time, the maximum number of instructions you should simulate is 1,000,000,00 for this task.

1. Total number of warps that executed conditional branch instructions and have divergence.
2. Total number of warps that executed conditional branches, no matter they have divergence or not.

Hint: For this task, read the `scheduler_unit` in shader.cc file and the `simt_stack` in abstract_hardware_model.h/.cc carefully and understand how they work. The benchmark you should use for this task is LPS. You need to dump out the counters and add them to your report. The output should look like the following,

```
# of warps excuted conditional branch instrctions and have divergence: XXX

# of warps excuted conditional branch instrctions(no matter they have divergence or not): XXX
```

3. Submission

- Create a directory named ”cuda-sim” and copy the files you modified for task1 to this directory.
- Create a directory named ”gpgpu-sim”, then create two subdirectories named ”task2” and ”task3”, and copy the files you modified for task2 and task3 respectively.
- Write a 1-2 page report briefly explaining how you completed each task and what you changed in the code. Make sure you include your IPC plot from the second task.
- Zip ”cuda-sim”, ”gpgpu-sim” and your report as <unityId.zip>
  - `zip -r unityId.zip cuda-sim gpgpu-sim report.pdf`

4. Cheating and LLMs

- You are supposed to do this assignment on your own. You can have discussions with your classmates, but we expect you to write your own code at the end of the day.
- Do not submit AI generated code/report.

Violation of these rules may result in zero marks and disciplinary action.